

Spike Inference

Using a Bi-Directional LSTM to predict spikes from LFP data

- Leonardo Ferrisi
- Alana Maluszczyk
- Daniel Feldman

Imports

```
import os
import numpy as np
import pandas as pd
import multiprocessing
import scipy.io as sio
import time
import matplotlib.pyplot as plt
from scipy.ndimage import gaussian_filter1d
import sklearn as sk
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from scipy.ndimage import gaussian_filter1d
from accuracy_metrics import threshold_accuracy, r_squared

# ----- Set Environment Variables -----
os.environ["TF_ENABLE_ONEDNN_OPTS"] = "0" # Disable oneDNN
optimizations (optional)
os.environ["TF_CPP_MIN_LOG_LEVEL"] = "2" # Suppress unnecessary
TensorFlow logs

# Determine the number of CPU cores available
num_cores = multiprocessing.cpu_count()
print("Number of CPU cores available:", num_cores)

# Set CPU parallelism
os.environ["OMP_NUM_THREADS"] = str(num_cores)
os.environ["TF_NUM_INTRAOP_THREADS"] = str(num_cores)
os.environ["TF_NUM_INTEROP_THREADS"] = str(num_cores)

# ----- TensorFlow GPU Setup -----
import tensorflow as tf
from keras.api.models import Sequential
from keras.api.layers import (
    Bidirectional, LSTM, Dropout, Dense, LayerNormalization
)
from keras.api.optimizers import Adam
```

```

import keras.api.backend as K
from keras.api.regularizers import l2
from alive_progress import alive_bar

# Check for available GPUs
gpus = tf.config.list_physical_devices("GPU")
if gpus:
    try:
        # Enable memory growth to prevent TensorFlow from consuming
        all GPU memory
        for gpu in gpus:
            tf.config.experimental.set_memory_growth(gpu, True)
        print(f"Using GPU: {gpus}")
    except RuntimeError as e:
        print(f"GPU setup error: {e}")
else:
    print("No GPU found, using CPU.")

# ----- Confirm TensorFlow Setup -----
print("TensorFlow version:", tf.__version__)
print("Num GPUs Available:",
      len(tf.config.list_physical_devices('GPU')))
print("TensorFlow intra-op threads:",
      tf.config.threading.get_intra_op_parallelism_threads())
print("TensorFlow inter-op threads:",
      tf.config.threading.get_inter_op_parallelism_threads())

#
=====
=====

```

Helper Methods

```

# Helper Methods
def create_sequences(signal, labels, window_size, batch_size=10000):
    """
    Create sequences from the 1D time series signal.
    For each window of LFP data, the label is the spike value at the
    time immediately after the window.
    """
    num_samples = signal.shape[1] - window_size
    X = np.memmap('X_data.dat', dtype=np.float32, mode='w+',
                  shape=(num_samples, window_size, signal.shape[0]))
    y = np.memmap('y_data.dat', dtype=np.float32, mode='w+',
                  shape=(num_samples,))
    # X, y = [], []
    with alive_bar(num_samples, title="Creating sequences") as bar:
        for i in range(num_samples):
            X[i] = np.transpose(signal[:, i : i + window_size]) #
    Transpose to match shape (timesteps, channels)

```

```

        y[i] = labels[i + window_size] # Label is the spike
        immediately after the window
        bar()
    return X, y

def load_data(data_path:str, data_key:str='Data', debug:bool=True) ->
pd.DataFrame:
    """
    Load data from a .mat file and return the data array as a pandas
    dataframe.

    @param data_path: Path to the .mat file
    @param debug: Print debug information
    @return: Pandas dataframe with the data
    """

    # Load the .mat file (adjust the filename as needed)
    mat_contents = sio.loadmat(data_path)

    # List all variable names in the file
    print(mat_contents.keys())

    # Replace 'data' with the actual variable name stored in your .mat
    file
    data_array = mat_contents[data_key]

    # Verify the shape and data type
    if debug:
        print("Data loaded from:", data_path)
        print("Shape:", data_array.shape)
        print("Data type:", data_array.dtype)
        print("-----")

    return pd.DataFrame(data_array)

def get_spike_firing_rate(spikes:pd.DataFrame, window_size:int|float,
debug_plot:bool=False) -> pd.DataFrame:
    """
    Calculate the spike firing rate from the binary spike array.

    @param spikes: Binary array of spike events (0 or 1)
    @param window_size: Size of the window to use for the convolution
    @return: Spike firing rate array
    """

    std = window_size // 4.0 # std deviation of the gaussian
    spike_firing_rate = gaussian_filter1d(spikes, truncate=4.0,
sigma=std)

    if debug_plot:
        fig, axs = plt.subplots(2, 1, figsize=(15, 10), sharex=True)

```

```

    # Plot the spike firing rate
    axs[0].plot(spike_firing_rate, color='blue')
    axs[0].set_title('Spike Firing Rate')
    axs[0].set_ylabel('Firing Rate')
    axs[0].grid()

    # Plot the spikes
    axs[1].plot(spikes, color='red')
    axs[1].set_title('Spikes')
    axs[1].set_xlabel('Time (ms)')
    axs[1].set_ylabel('Spikes')
    axs[1].grid()

    plt.tight_layout()
    plt.show()

    return spike_firing_rate

def clean_up_memmap_files(X_memmap, y_memmap, delete_files=True):
    """
    Clean up memory-mapped files.

    Parameters:
    X_memmap: The memory-mapped X data
    y_memmap: The memory-mapped y data
    delete_files: Whether to delete the underlying files from disk
    """
    # Close the memmap objects to ensure all changes are written to
    disk
    if hasattr(X_memmap, '_mmap') and X_memmap._mmap is not None:
        X_memmap._mmap.close()

    if hasattr(y_memmap, '_mmap') and y_memmap._mmap is not None:
        y_memmap._mmap.close()

    # Delete the files if requested
    if delete_files:
        import os
        if os.path.exists('X_data.dat'):
            os.remove('X_data.dat')
            print("Deleted X_data.dat")

        if os.path.exists('y_data.dat'):
            os.remove('y_data.dat')
            print("Deleted y_data.dat")

def rmse(y_true, y_pred):
    """
    Calculates the Root Mean Squared Error between y_true and y_pred

```

using TensorFlow operations.

Parameters:

y_true (tensor): True target values.

y_pred (tensor): Predicted values.

Returns:

tensor: The root mean squared error.

"""

return tf.math.sqrt(tf.reduce_mean(tf.square(y_pred - y_true)))

Main code for running spike inference

```
def spike_inference(spikes_file, lfp_file, lfp_key='Data',
spikes_key='spikes_1k', region='NA', samples_ms=20000, lfp_channel=1,
debug:bool=False, debug_lfp:bool=False, render_logo=False,
shuffle_validation=False):
    """
    Perform spike inference using LFP (Local Field Potential) data and
    spike times.
    This function loads spike and LFP data, preprocesses the data,
    standardizes the signals,
    creates sequences for LSTM (Long Short-Term Memory) model
    training, builds and trains
    a bidirectional LSTM model, and plots the training history.

    Parameters:
        spikes_file : str
            Path to the file containing spike times data.
        lfp_file : str
            Path to the file containing LFP data.
        lfp_key : str, optional
            Key to access LFP data within the file (default is
'Data').
        spikes_key : str, optional
            Key to access spike times data within the file (default is
'spikes_1k').
        region : str, optional
            Region of interest (default is 'NA').
        samples_ms : int, optional
            Number of samples in milliseconds to use for testing
(default is 20000).
        lfp_channel : int, optional
            LFP channel to use for analysis (default is 1).
        debug : bool, optional
            If True, print debug information (default is False).
        debug_lfp : bool, optional
            If True, plot the standardized LFP signal for debugging
(default is False).
```

```

        render_logo : bool, optional
            If True, render the logo using the qol package (default is
False).
        shuffle_validation : bool, optional
            If True, shuffle the validation data during splitting
(default is False).
        *Returns*: None

-----
Notes:
- The function saves the trained Keras model to an H5 file with a
timestamp.
- The function plots the training and validation loss and MSE
(Mean Squared Error) after training.
"""
if render_logo: from qol import render_logo; render_logo()

# -----
# 1. Data Loading and Preprocessing
# -----
# Load the spikes data
if debug: print("-----\nLoading
data:\n")
spikes_1k_df = load_data(spikes_file, data_key=spikes_key)
# **Where spikes_1k_df is a 132x4983702 array, where there are
132 channels and 4983702 time points

sEEG_df = load_data(lfp_file, data_key=lfp_key)

# Convert spike_times into a gaussian firing rate
-----
spikes_times = spikes_1k_df.values[0]

# Create logical array of size (1, lfp.shape[1]) of zeros
# ms_buffer = 1000 # 1s buffer after last timestamp

spikes_times = np.round(spikes_times).astype(np.int32) # Convert
to int for indexing

spikes = np.zeros(len(sEEG_df.values[0]))

# For each spike time in spike_times, set that index in spikes
to 1
for spike_time in spikes_times:
    spikes[spike_time] = 1

spikes_firing_rate = get_spike_firing_rate(spikes,
window_size=10000, debug_plot=False)

if debug: print("-----\n")

```

```

# -----
# 2. Standardize Signals
# -----
# Handle LFP
# sEEG_df is a 132x4983702 array, where each row is a channel and
each column is a time point.
# For initial testing, use only a subset of the data.
# max_samples = sEEG_df.shape[1]
max_samples = samples_ms # Use a subset of samples for testing
num_channels = 1 # Use a subset of channels for testing (max of
132)

# TODO: UPDATE: We dont actually know which is the nearest LFP
sEEG contact point on the electrode. Using the first one for now...

# Standardize each channel (axis=1) using StandardScaler:
# Initialize 1D array for the single channel
lfp = sEEG_df.values[lfp_channel, :max_samples].astype(np.float32)

# Standardize the 1D signal
scaler_lfp = StandardScaler()
lfp = scaler_lfp.fit_transform(lfp.reshape(-1, 1)).ravel()

# Reshape to (1, samples) to maintain expected dimensions for
later processing
lfp = lfp.reshape(1, -1)

# Plotting standardized LFP signal as a sanity check
if debug_lfp:
    plt.figure(figsize=(10, 4))
    for i in range(1):
        plt.plot(lfp[i, :1000], label=f'Channel {i+1}') # Plot
first 1000 samples
    plt.title('Standardized LFP Signal')
    plt.xlabel('Time (samples)')
    plt.ylabel('Amplitude (standardized)')
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.show()

# Standardize spikes firing rate signal
spikes_firing_rate =
spikes_firing_rate[:max_samples].astype(np.float32)
scaler_spikes = StandardScaler()
spikes_standardized =
scaler_spikes.fit_transform(spikes_firing_rate.reshape(-1, 1)).ravel()

# -----

```

```

# 3. Creating Sequences for the LSTM
# -----
window_size = 50 # window size in timesteps
X, y = create_sequences(lfp, spikes_standardized, window_size)

# Reshape X to (samples, timesteps, features)
if debug:
    print("Before reshaping:")
    print(f"X shape: {X.shape}, Expected: (num_samples, {window_size}, {num_channels})")
    print(f"y shape: {y.shape}, Expected: (num_samples,)")
X = X.reshape(-1, window_size, num_channels)
if debug:
    print("After reshaping:")
    print(f"X shape: {X.shape}")
    print(f"y shape: {y.shape}")

# -----
# 4. Splitting the Dataset: 70% Training, 30% Validation
# -----
X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=0.3, random_state=42,
    shuffle=shuffle_validation
)
if debug:
    print(f"Training set shape: X_train={X_train.shape}, y_train={y_train.shape}")
    print(f"Validation set shape: X_val={X_val.shape}, y_val={y_val.shape}")

# -----
# 5. Building the Bidirectional LSTM Model for Regression
# -----
input_timesteps = X_train.shape[1]
input_features = X_train.shape[2]
# Create a more robust model with better regularization
model = Sequential([
    # First Bidirectional LSTM layer with LayerNormalization
    # followed by dropout
    Bidirectional(LSTM(64, return_sequences=True),
    input_shape=(input_timesteps, input_features)),
    LayerNormalization(),
    Dropout(0.3),

    # Second Bidirectional LSTM layer with LayerNormalization
    # followed by dropout
    Bidirectional(LSTM(64, return_sequences=False)),
    LayerNormalization(),
    Dropout(0.3),

```



```

        # Final Dense layer for regression (predicting a continuous
value)
        Dense(1, activation='linear')
    ])

    # Use a lower learning rate for better convergence
    optimizer = Adam(learning_rate=0.00005)

    time_start_model = time.time()
    model.compile(
        loss='mean_squared_error',
        optimizer=optimizer,
        metrics=['mse', rmse, r_squared, 'mae']
    )
    model.summary()
    time_end_model = time.time()
    print(f"Model building time: {time_end_model - time_start_model}
seconds")

# -----
# 6. Training the Model
# -----
    time_start_training = time.time()
    history = model.fit(
        X_train, y_train,
        validation_data=(X_val, y_val),
        epochs=50,          # Adjust the number of epochs as needed
        batch_size=10,      # Adjust batch size as needed
        verbose=1
    )
    time_end_training = time.time()
    print(f"Model training time: {time_end_training -
time_start_training} seconds")

    # Save the Keras model to an H5 file with a timestamp
    from datetime import datetime
    timestamp = datetime.now().strftime("%Y%m%d_%H%M")
    os.makedirs("models", exist_ok=True)
    keras_model_path = f"models/spike_inference_model_{timestamp}.h5"
    model.save(keras_model_path)
    print(f"Keras model saved to {keras_model_path}")

    # Clean up memory mapped files after model is trained
    clean_up_memmap_files(X, y, delete_files=True)

# -----
# 7. Plotting Training History
# -----
    print(f"History keys are: {history.history.keys()}")

```

```

plt.figure(figsize=(10, 5))
plt.plot(history.history['loss'], label='Training Loss',
color='blue')
plt.plot(history.history['val_loss'], label='Validation Loss',
color='red')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.title('Training & Validation Loss')
plt.legend()
plt.grid()
plt.show()

plt.figure(figsize=(10, 5))
plt.plot(history.history['mse'], label='Training MSE',
color='blue')
plt.plot(history.history['val_mse'], label='Validation MSE',
color='red')
plt.xlabel('Epochs')
plt.ylabel('MSE')
plt.title('Training & Validation MSE')
plt.legend()
plt.grid()
plt.show()

```

Run Spike Inference

```

spike_inference(lfp_channel=65, samples_ms=250000,
spikes_file='data/actual_data/patient1/spike_times_set1_1k.mat',
spikes_key='spike_times_set1_1k',
lfp_file='data/actual_data/patient1/try_sEEG_Data.mat',
lfp_key='Data',
region='NA', debug=True, render_logo=False,
shuffle_validation=False)

```